

Base de suivi des prédictions

tracking.db — structure et contenu actuel

DeepEdge Benchmark · Partie B · état au 8 juillet 2026

1. À quoi sert cette base

La base tracking.db est le journal de bord des prévisions produites par les modèles du benchmark. Chaque fois qu'un modèle prédit le prix futur d'un actif, la prédiction y est enregistrée. Plus tard, quand le vrai prix est connu, la base compare la prédiction à la réalité et calcule automatiquement si le modèle a été juste. Elle sert donc à **mesurer objectivement la qualité de chaque modèle** dans la durée.

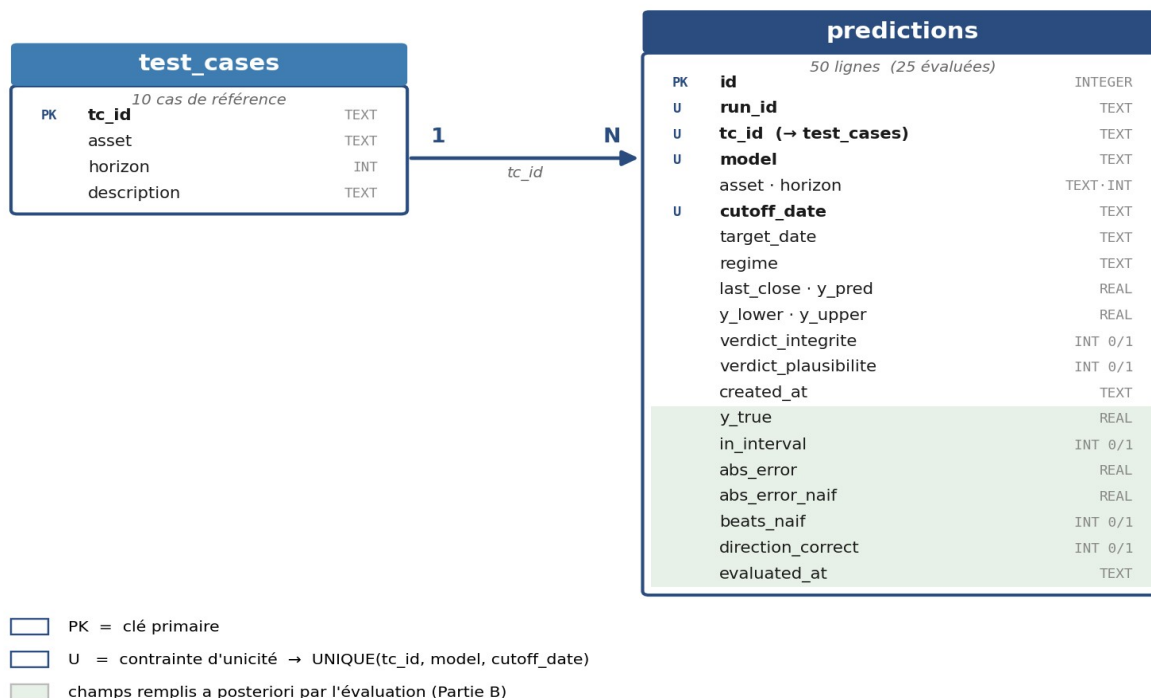
Techniquement, c'est une base **SQLite** — un simple fichier (validation/tracking.db, 32 Ko), sans serveur, lisible avec la bibliothèque standard de Python. Elle est volontairement légère et autonome.

Le principe sépare deux moments : **(1) la production** — un modèle écrit sa prédiction — et **(2) l'évaluation** — une fois l'échéance passée, on récupère le vrai prix et on note la prédiction. C'est ce qui explique la structure de la table principale, décrite plus bas.

2. Schéma d'ensemble

La base contient **deux tables** reliées par le champ tc_id : test_cases (le catalogue des cas testés) et predictions (toutes les prévisions et leur évaluation).

tracking.db — schéma relationnel (SQLite)



Lecture : une ligne de `test_cases` (un cas de référence) correspond à plusieurs lignes de `predictions` (une par modèle et par run). Les champs sur fond vert clair ne sont remplis qu'après coup, lors de l'évaluation.

3. Table `test_cases` — le catalogue des cas

Elle liste les **cas de référence** que l'on cherche à prédire : un actif + un horizon. Table courte et stable, elle sert de dictionnaire.

Champ	Type	Clé	Signification
tc_id	TEXT	PK	Identifiant du cas de test (ex. TC_SPY_D1)
asset	TEXT		Ticker de l'actif (SPY, BTC-USD...)
horizon	INTEGER		Horizon de prévision : 1 ou 7 jours
description	TEXT		Libellé libre du cas

4. Table `predictions` — le cœur de la base

Chaque ligne est **une prévision faite par un modèle donné, pour un cas donné, à une date de coupure donnée**. Les champs se lisent en deux groupes.

Groupe 1 — écrit à la création (dès que le modèle prédit) : de `run_id` à `created_at`. On y trouve la prévision (`y_pred`) et son intervalle de confiance (`y_lower` / `y_upper`).

Groupe 2 — écrit à l'évaluation (lignes vertes, une fois le vrai prix connu) : de `y_true` à `evaluated_at`. Tant que l'échéance n'est pas passée, ces champs restent vides.

Champ	Type	Clé	Signification
id	INTEGER	PK	Clé technique auto-incrémentée
run_id	TEXT	U	Identifiant du run de benchmark
tc_id	TEXT	U	→ référence <code>test_cases.tc_id</code>
model	TEXT	U	arima-garch / sarima / prophet / lstm / naive
asset	TEXT		Ticker de l'actif
horizon	INTEGER		1 ou 7 jours de bourse
cutoff_date	TEXT	U	Date de coupure T (dernière donnée d'entraînement)
target_date	TEXT		Date prédite T+H
regime	TEXT		Régime de marché à T (bear / calm)
last_close	REAL		Prix à T = prévision naïve de référence
y_pred	REAL		Prévision ponctuelle du modèle

Champ	Type	Clé	Signification
y_lower	REAL		Borne basse de l'intervalle de confiance 95 %
y_upper	REAL		Borne haute de l'intervalle de confiance 95 %
verdict_integrite	INTEGER		Verdict niveau 1 (0/1), calculé en amont
verdict_plausibilite	INTEGER		Verdict niveau 2 (0/1), calculé en amont
created_at	TEXT		Horodatage de création de la ligne
y_true	REAL	éval.	Vraie valeur observée à target_date
in_interval	INTEGER	éval.	1 si y_true dans [y_lower, y_upper]
abs_error	REAL	éval.	Erreur absolue du modèle
abs_error_naif	REAL	éval.	Erreur absolue de la prévision naïve
beats_naif	INTEGER	éval.	1 si le modèle bat le naïf
direction_correct	INTEGER	éval.	1 si le sens de variation est correct
evaluated_at	TEXT	éval.	Date de l'évaluation a posteriori

La règle anti-doublon

Une contrainte d'unicité porte sur le trio (tc_id, model, cutoff_date) : **un même modèle ne peut pas enregistrer deux prévisions pour le même cas à la même date de coupure**. Si le benchmark est relancé, les lignes déjà présentes sont ignorées plutôt que dupliquées — la base reste propre (idempotence).

5. Le cycle de vie d'une prédiction

1. **Création.** Le modèle prédit ; une ligne est insérée avec sa prévision et son intervalle. Les champs d'évaluation sont vides.
2. **Attente.** On attend que la date cible (target_date) soit passée et que le vrai prix soit disponible.
3. **Évaluation.** Un traitement récupère le prix réel, le stocke dans y_true, puis calcule les cinq indicateurs de qualité et les inscrit dans la ligne.

Cette évaluation tourne automatiquement chaque jour, ce qui explique qu'à un instant donné une partie seulement des lignes soit évaluée (voir §7).

6. Les indicateurs de qualité calculés

Sur un prix quasi imprévisible, une erreur brute ne dit pas grand-chose. La base mesure donc surtout la **calibration** et le **skill face au naïf** :

Indicateur	Ce qu'il mesure
in_interval	Le vrai prix tombe-t-il dans l'intervalle de confiance 95 % ? (fiabilité de l'incertitude annoncée)
abs_error	Erreur absolue du modèle : vrai – prévu
abs_error_naïf	Erreur de la prévision naïve (« demain = aujourd'hui »)
beats_naïf	Le modèle fait-il mieux que le naïf ? (le vrai test de valeur ajoutée)
direction_correct	Le sens de variation (hausse/baisse) est-il correct ?

7. Contenu actuel de la base

État au 8 juillet 2026 — un seul run à ce jour (run_20260707T111806, coupure du 6 juillet) :

Élément	Valeur
Cas de référence (test_cases)	10
Prévisions enregistrées (predictions)	50
Prévisions déjà évaluées	25 (horizon 1 jour, échéance atteinte)
Prévisions en attente	25 (horizon 7 jours, échéance au 13 juillet)
Modèles couverts	5 : ARIMA-GARCH, SARIMA, Prophet, LSTM, Naïve
Actifs couverts	5 : BTC-USD, ETH-USD, SPY, TLT, ZN=F
Horizons	2 : 1 jour et 7 jours
Régimes de marché observés	bear (crypto) et calm (SPY, TLT, ZN=F)

Chaque combinaison **5 modèles × 5 actifs × 2 horizons** donne les 50 lignes. Les 25 lignes à horizon 1 jour sont déjà notées ; les 25 à horizon 7 jours le seront après le 13 juillet.

Aperçu des premiers résultats (25 lignes évaluées, horizon 1 jour)

Échantillon très réduit (5 prévisions par modèle) : à lire comme une illustration de ce que produit la base, pas comme un classement.

Modèle	Couvre. IC	Bat naïf	Sens OK	sur n
ARIMA-GARCH	5/5	1/5	1/5	5
SARIMA	5/5	2/5	2/5	5
Prophet	4/5	0/5	0/5	5
LSTM	5/5	1/5	2/5	5

Modèle	Couvre. IC	Bat naïf	Sens OK	sur n
Naïve	5/5	2/5	3/5	5

« Couvre. IC » = nb de fois où le vrai prix tombe dans l'intervalle 95 %. « Bat naïf » = nb de fois où le modèle fait mieux que la prévision naïve.

En résumé

- Une base SQLite d'un seul fichier, validation/tracking.db.
- Deux tables : test_cases (10 cas) et predictions (50 prévisions).
- Chaque prévision est d'abord enregistrée, puis évaluée automatiquement une fois le vrai prix connu.
- Une contrainte d'unicité garantit qu'aucune prévision n'est comptée deux fois.
- À ce jour : 1 run, 25 lignes évaluées sur 50, 5 modèles, 5 actifs, 2 horizons.